Addendum — Local/Global World-Model Planning with DINO-WM: A Decoupled Forward–Backward Planning Track

Addendum to *DINO-WM on PointMaze under a Commodity-GPU Budget*; implementation and protocol, results pending

Thomas Georges*

June 2026 — addendum, draft v0.1

This is an addendum to the main report "DINO-WM on PointMaze under a Commodity-GPU Budget" (`reports/dino_wm_report.tex`). That report documents the DINO-WM model, its latent-cached training, and the planning results this track builds on; its DINO-WM background and notation are shared here and not re-derived.

Abstract

Accurate world models and differentiable world models pull in different directions: the models that produce the most trustworthy forward rollouts (large latent transformers, diffusion world models) are expensive or impractical to differentiate through, while small differentiable surrogates are easy to optimize against but easy to exploit. Following the decoupled forward–backward idea of coupled local/global world models, we add a *local/global planning track* to an existing DINO-WM reproduction: the frozen DINO-WM latent world model serves as the trusted *global* forward simulator and scorer, a small residual surrogate trained on cached DINOv2 patch latents serves as the *local* differentiable model, and a family of planners spans the spectrum between zero-order global search (CEM) and first-order local optimization (gradient descent through the surrogate), including hybrid planners in which global search proposes, local gradients refine, and a global re-score accepts or rejects the refinement. We describe the method, the offline latent goal-reaching evaluation protocol with its fairness accounting, the safeguards against surrogate exploitation, and the engineering required to run the full, un-watered-down experiment on freely available hardware (Colab T4) with resumable training and evaluation. Quantitative PointMaze results will be added in a subsequent revision.

1 Introduction

Model-predictive control with learned world models faces a structural trade-off. Zero-order planners such as the cross-entropy method (CEM) only require forward rollouts and therefore work with arbitrarily large and accurate models, but their sample complexity grows quickly with horizon and action dimensionality. First-order planners obtain gradients of the planning objective with respect to actions and can be far more sample-efficient per iteration, but they require a differentiable model — and when that model is small enough to differentiate cheaply,

*Implementation assistance: Claude (Anthropic). Code: `wm-prediction` repository, `src/wm_poc/local_global`. This is an addendum to the main report *DINO-WM on PointMaze under a Commodity-GPU Budget: Latent-Cached Reproduction, Low-Data Transfer, and Planning Results* (`reports/dino_wm_report.tex`), whose trained checkpoint and latent cache this track reuses unchanged.

it is usually also inaccurate enough to be *exploited*: gradient descent happily walks into regions where the surrogate is wrong.

The coupled local/global world-model line of work [1] resolves the trade-off by decoupling the two roles. A large, accurate *global* model produces forward trajectories and is never differentiated; a lightweight *local* model supplies derivatives, evaluated along the global model's trajectories, so that the local model only ever needs single-step accuracy in the neighborhood of states the global model visits. We transplant this idea into a simpler, fully observable setting — visual goal reaching in latent space with DINO-WM [2] — where it becomes a statement about *planners* rather than policy learning:

- the **global model** is the repository's existing DINO-WM latent world model (a frozen ViT predictor over frozen DINOv2 patch features [3]), used as forward simulator and scorer, always under `no_grad`;
- the **local model** is a small residual MLP (or GRU) surrogate operating on a compressed projection of the same latents, trained from cached transitions, fully differentiable with respect to actions;
- **planners** combine the two: global CEM, local first-order (GD/Adam), local CEM (an ablation isolating model error from optimizer effects), and hybrid CEM + local refinement with an optional global re-score that can reject harmful refinements.

Contributions. This report documents (i) a concrete local/global planning instantiation on top of DINO-WM, including the surrogate architecture and training objective (§4); (ii) an offline latent goal-reaching evaluation protocol with explicit fairness accounting and honestly stated biases (§5); (iii) three safeguards against surrogate exploitation and a measurement of local–global disagreement (§4.5); and (iv) the engineering that makes the full experiment run, resume, and complete on commodity hardware (§6). Results tables and figures are left as placeholders (§8) to be filled once the first full PointMaze run completes.

2 Background

DINO-WM. DINO-WM [2] trains an action-conditioned latent transition model directly on frozen DINOv2 patch features $\mathbf{z}_t \in \mathbb{R}^{P\times D}$ (P=196 patches, D=384 for ViT-S/14 at 224 px), with teacher-forced latent-consistency training and no pixel reconstruction in the planning loop. Planning is visual goal reaching: given a current and a goal observation, CEM/MPC minimizes the latent distance between the predicted future state and the goal's encoding. The main report (*DINO-WM on PointMaze under a Commodity-GPU Budget*, `reports/dino_wm_report.tex`) reproduces DINO-WM training on PointMaze with a precomputed latent cache; this addendum reuses that trained checkpoint and cache unchanged.

Decoupled forward/backward models. The FOG line of work [1] trains policies by differentiating short-horizon returns, substituting the Jacobians of a lightweight RSSM for the true dynamics derivatives while the trajectory itself is generated by a large frozen diffusion world model. Two findings motivate our design: the small model alone is insufficient as a forward simulator (their "no diffusion" ablation fails), and the small model's derivatives are most useful when evaluated *at states produced by the trusted model*. Our MPC loop mirrors both: the global model is the only simulator, and the surrogate re-anchors on globally imagined latents at every replanning step.

Related planning paradigms. PlaNet [4] and Dreamer [5] plan or learn policies inside a learned recurrent latent model; TD-MPC2 [7] combines a learned latent model with sampling-based MPC and a value prior; differentiable MPC [8] differentiates through the optimizer itself. V-JEPA 2 [9] and the JEPA world-model family [10] support the broader pattern of planning on top of frozen self-supervised representations that this track inherits.

3 Problem setting and notation

Episodes are sequences of latents $\mathbf{z}_0, \ldots, \mathbf{z}_{T-1}$ ($\mathbf{z}_t \in \mathbb{R}^{P\times D}$, cached fp16) with raw actions $\mathbf{a}_0, \cdots \in \mathbb{R}^A$ (A=2 for PointMaze, bounds $[-1,1]^A$). Following DINO-WM, one *model step* advances f raw steps (frameskip f=5): the model-step action is the folded block

$$\bar{\mathbf{a}}_k \;=\; \big[\mathbf{a}_{t_0+kf};\, \mathbf{a}_{t_0+kf+1}; \ldots; \mathbf{a}_{t_0+(k+1)f-1}\big] \in \mathbb{R}^{fA}, \tag{1}$$

and latent frames are sampled at $t_0, t_0{+}f, t_0{+}2f, \ldots$ Training windows consist of C context frames and K rollout target frames (C=2, K=3), with the $C{-}1{+}K$ action blocks between consecutive frames; a window starting at t_0 is valid iff $t_0 \le \min\big(T_z - 1 - (C{+}K{-}1)f,\; T_a - (C{+}K{-}1)f\big)$. Train/validation splits are by episode (never by window), drawn from a seeded permutation of the *uncapped* episode count so that membership is invariant to smoke-test subsampling.

A *goal-reaching task* is a tuple (episode, t_0): the planner observes the context window ending at t_{cur} and must reach the latent $\mathbf{z}_{\text{goal}} = \mathbf{z}_{t_{\text{cur}}+Gf}$ recorded G model steps later in the same episode (G=5).

4 Method

4.1 Global forward model

The global model is the trained DINO-WM checkpoint, loaded in-process exactly as the upstream planner loads it (Hydra train config + epoch checkpoint) and driven on cached latents through the repository's latent-bypass patch, so rollouts stay in latent space end to end. Three details matter:

1. **Replay from an observed anchor.** The upstream rollout API expects one action block per context frame followed by future blocks. The adapter keeps the original *observed* context (latents, raw proprioception, and the $C{-}1$ observed action blocks) plus the list of executed blocks, and replays from that anchor at every call. The model is therefore never conditioned on fabricated proprioception for imagined frames.

2. **Normalization at the boundary.** Checkpoints trained with upstream action/proprio normalization receive normalized inputs inside the adapter (statistics recomputed from the raw dataset exactly as upstream computes them); every other component works in raw units.

3. **Chunked candidate scoring.** CEM populations are scored in chunks of `rollout_batch_size` candidates. Candidates are independent, so chunking provably does not change scores or per-candidate call counts — only peak memory — which is what allows the identical experiment to run on a 16 GB T4 (chunk 32) and an A100 (chunk 128).

All global-model calls execute under `torch.no_grad()`; gradients can only originate from the surrogate. For CPU-only tests and smokes, a synthetic point-mass task with *exactly* linear latents ($\mathbf{z} = W\mathbf{s} + b$) provides a closed-form perfect global model (decode with the pseudo-inverse, step the true dynamics, re-encode), so planner correctness is measured against true dynamics rather than a learned proxy.

4.2 Local surrogate

Projection. Global latents are compressed by $\mathbf{x} = W_p\,\mathrm{pool}(\mathrm{LN}(\mathbf{z})) \in \mathbb{R}^d$ (d=256), where pool is either a global patch mean or a 4×4 spatial grid average (preserving coarse object layout), and W_p is a *frozen*, seeded, semi-orthogonal matrix. Freezing is principled rather than incidental: the training loss is an MSE *in projected space* between predictions and projected targets, for which a jointly trained projector admits the degenerate optimum W_p=0 (representation collapse). A frozen near-orthogonal projection of LayerNormed features preserves distances in the Johnson–Lindenstrauss sense and removes the collapse mode entirely; a trainable variant exists behind a flag and requires an accompanying hinge variance penalty.

Dynamics. The default surrogate is a residual MLP, $\mathbf{x}_{k+1} = \mathbf{x}_k + g_\theta([\mathbf{x}_k; \bar{\mathbf{a}}_k])$ (LayerNorm input, SiLU, hidden width 512, 3 layers), so the identity map is the zero-function default. A GRU variant consumes the $C-1$ context transitions with a single `GRUCell` to build a hidden state (velocity inference) and predicts residual deltas from it. Rollouts unroll the step function under autograd; horizons are short (≤ 6), so exact backpropagation through time is cheap.

4.3 Surrogate training objective

With $\hat{\mathbf{x}}_{1:K}$ the surrogate rollout from the context and $\mathbf{x}_{1:K}$ the projections of cached target latents,

$$\begin{aligned}\mathcal{L} \;=\; & \lambda_{\text{roll}} \frac{\sum_{k=1}^{K} \gamma^{k-1} \,\|\hat{\mathbf{x}}_k - \mathbf{x}_k\|_2^2/d}{\sum_{k=1}^{K} \gamma^{k-1}} \;+\; \lambda_1 \,\|\hat{\mathbf{x}}_1 - \mathbf{x}_1\|_2^2/d \\ & +\; \lambda_\Delta \,\|(\hat{\mathbf{x}}_1 - \mathbf{x}_0) - (\mathbf{x}_1 - \mathbf{x}_0)\|_2^2/d \;+\; \lambda_J \left\|\partial \hat{\mathbf{x}}_1 / \partial \bar{\mathbf{a}}_0\right\|_2^2 \;+\; \lambda_V \sum_j \max(0, 1 - \sigma_j(\mathbf{x})), \quad (2)\end{aligned}$$

i.e. a (optionally discounted) multi-step rollout MSE, a one-step term, a *delta* term that keeps predicted state changes calibrated when absolute error is dominated by static features, an optional action-Jacobian penalty (double backward) reserved for taming exploitable gradients, and the variance hinge used only with a trainable projector. Defaults: λ_{roll}=λ_1=1, λ_Δ=0.1, λ_J=λ_V=0, γ=1. Training reports a scale-free diagnostic, the rollout MSE divided by the "predict no change" baseline; values well below 1 certify the surrogate beats the trivial predictor independently of the projection's numeric scale.

4.4 Planners

All planners optimize an open-loop sequence $\bar{\mathbf{a}}_{1:H} \in \mathbb{R}^{H \times fA}$ toward $\mathbf{z}_{\text{goal}}$, minimizing

$$J(\bar{\mathbf{a}}_{1:H}) \;=\; \underbrace{\tfrac{1}{PD}\|\hat{\mathbf{z}}_H(\bar{\mathbf{a}}_{1:H}) - \mathbf{z}_{\text{goal}}\|_2^2}_{\text{goal cost}} + \lambda_{\text{sm}} \underbrace{\tfrac{1}{(H-1)fA} \textstyle\sum_k \|\bar{\mathbf{a}}_{k+1} - \bar{\mathbf{a}}_k\|_2^2}_{\text{smoothness}}, \quad (3)$$

where $\hat{\mathbf{z}}_H$ is the model's predicted final latent (global model for global planners; for local planners the analogous cost is computed in projected space against $\mathbf{x}_{\text{goal}}$). Planners optimize J but *report* goal and smoothness components separately, so the smoothness weight cannot contaminate cross-planner distance comparisons.

Global CEM. Iterations sample a population of N sequences from a diagonal Gaussian (initial std expressed in units of half the action range), clamp to bounds, score with the global model, and refit mean and std to the top-E elites with a std floor; the incumbent mean is always kept in the pool, and the best-ever candidate (not the final mean) is returned. Sampling uses a CPU generator seeded per planning round, making results device-independent and reproducible.

Local first-order (GD/Adam). Actions are parameterized through a differentiable squash, $\bar{\mathbf{a}} = \ell + (u - \ell)\,\frac{1}{2}(\tanh(\mathbf{r}) + 1)$, so iterates are feasible by construction and gradients respect the boundary geometry. The raw parameters are optimized by SGD or Adam with gradient-norm clipping, and the best iterate (not the last) is returned. Warm starts from CEM solutions require care: CEM clamps samples *onto* the bounds, and $\operatorname{atanh}(1 - 10^{-5})$ sits where the tanh derivative is $\approx 2\times10^{-5}$, freezing refinement. Warm starts therefore unsquash with a dedicated margin $\varepsilon_{\text{ws}}=10^{-2}$ (derivative ≈ 0.02), trading a $\leq 1\%$ bound offset for usable gradients.

Hybrid CEM + local refinement (+ global re-score). Global CEM proposes $\bar{\mathbf{a}}^{\text{CEM}}$ with global total cost J^{CEM}; the local planner refines from that warm start to $\bar{\mathbf{a}}^{\text{ref}}$; the global model re-scores the refinement *on the same objective*, yielding J^{ref}. With re-scoring enabled, the refinement is rejected iff

$$J^{\text{ref}} \;>\; J^{\text{CEM}} + \tau\, \max(|J^{\text{CEM}}|, \epsilon), \qquad \tau = 0.05, \tag{4}$$

in which case the CEM sequence is returned unchanged. Both "refinement improved the global cost" and "refinement accepted" are logged per planning round. A *local CEM* planner (CEM scored by the surrogate) completes the 2×2 design (model × optimizer), isolating the "small model" effect from the "gradients" effect.

Compute accounting. Every planner counts global forward rollouts *per candidate* (a CEM round of population N over I iterations costs $NI{+}1$ global calls, the $+1$ being the final pure-goal re-evaluation), local forward calls, backward steps, and wall time. These are first-class columns of the results table: the local/global thesis is an *efficiency* claim, and the accounting is the currency it is paid in.

4.5 Safeguards against surrogate exploitation

Three safeguards are active by default (per the implementation specification), with the spec's optional extensions deferred: (1) *differentiable action bounds* via tanh squashing — the first-order planner cannot leave the feasible set, eliminating the most common exploitation channel; (2) an *action smoothness penalty* in the optimized objective; (3) *global re-scoring with rejection* (Eq. 4) — the surrogate is never trusted to overrule the global model. The evaluation additionally measures the surrogate's open-loop *disagreement*: the executed action sequence is replayed through the surrogate and compared, step by step in projected space, against the global model's imagined trajectory.

5 Evaluation protocol

Offline latent goal reaching. For each task, an MPC loop plans $H' = \min(H, \text{steps remaining})$ blocks (the horizon shrinks toward the goal), executes $m{=}1$ block on the *global model as simulator*, appends the imagined latent to the planning context, and repeats for G steps. Final performance is the global-latent distance to the goal. Per episode we record:

- **normalized final distance** $\rho = \|\mathbf{z}_{\text{fin}} - \mathbf{z}_{\text{goal}}\|^2 / \|\mathbf{z}_{\text{start}} - \mathbf{z}_{\text{goal}}\|^2$, and **success** $= \mathbb{1}[\rho < 0.5]$ ("closed at least half the gap"), making thresholds scale-free across encoders;
- **reference replay**: the dataset's true action block sequence from start to goal replayed through the same simulator — an achievable-by-construction calibration line (near zero under the exact synthetic model; its magnitude under DINO-WM measures the global model's own rollout error);
- local-space final distance, the disagreement metric of §4.5, refinement acceptance rate, bound-violation count (structurally zero), the three compute counters, and wall time.

Stated bias. Because the global model is simulator *and* judge, `global_cem` is structurally favored on distance metrics: it optimizes exactly the quantity it is scored on. The protocol is therefore read as follows: distance/success columns establish that local and hybrid planners are *competent*; the efficiency columns (wall time, call counts) and the acceptance rate carry the comparative claim. Confirmation in the real environment (MuJoCo PointMaze, via the host repository's upstream planning path) is explicitly listed as the next step and is out of scope for this revision.

6 Implementation

The track follows the host repository's two-notebook convention: an operational notebook (03) for setup verification, transition export, surrogate training, and planner evaluation — every heavy step delegated to a gated CLI script — and a read-only results notebook (08) that aggregates artifacts into tables and figures. Engineering features relevant to reproducing the experiment:

- **Single experiment definition, hardware-only profiles.** The A100 config file defines the experiment (model sizes, optimizer budgets, episode counts, run name); the T4 config extends it and overrides *throughput knobs only* (rollout chunk size, dataloader workers), inheriting the run name so a run started on one GPU tier resumes on another.
- **Resumability everywhere.** Surrogate training checkpoints model, optimizer state, and step counter every 1000 steps (plus best-by-validation every 500) onto persistent storage, and resumes by default; planning evaluation is wall-clock capped and resumes at per-episode granularity (the i-th task is deterministic in the seed), so the 100-episode sweep completes across sessions. An interruption costs at most 1000 training steps or one episode.
- **Determinism.** Episode splits, task sampling, CEM noise, and the frozen projection are all seeded; CEM sampling uses a CPU generator so results are identical across devices; chunked scoring is exactness-tested.
- **Testing.** ∼170 unit/integration tests run with torch (and a ∼130-test subset without it), including analytic toy planner tests — notably a sign-flipped surrogate that makes local refinement provably harmful, asserting the rejection gate fires — plus an end-to-end CPU smoke on the synthetic task covering all six planners in under two minutes.

7 Experimental setup

Task and data. PointMaze with the host repository's DINO-WM setup: frozen DINOv2 ViT-S/14 at 224 px (P=196, D=384), frameskip f=5, raw action bounds $[-1, 1]^2$; cached latents for ∼2200 episodes; episode-level validation split fraction 0.1, seed 42.

Component	Setting	Component	Setting
Projection	grid pool 4×4, frozen, d=256	CEM population / elites / iters	300 / 30 / 10
Dynamics	residual MLP, width 512, 3 layers	CEM init. std	0.5× half-range
Context / rollout	C=2, K=3	GD/Adam iters, lr	100, 0.05
Batch / steps / lr	512 / 20 000 / 3×10^{-4}	Horizon / goal steps / exec	5 / 5 / 1
Weight decay	10^{-4} (AdamW)	Smoothness λ_{sm}, tolerance τ	0.01, 0.05
Loss weights	λ_{roll}=λ_1=1, λ_Δ=0.1	Episodes / success threshold	100 / ρ<0.5

Table 1: Full-experiment configuration (`pointmaze_surrogate_a100.yaml`; the T4 profile differs only in rollout chunk size and dataloader workers).

Budget. Surrogate training is data-bound (each step moves ∼375 MB of cached latents), estimated at 40–80 min on A100 and 1–2.5 h on T4 with a 300-minute per-session cap and cross-session resume; the six-planner evaluation is dominated by global CEM rollouts and is similarly capped and resumable. The episode count is set to 100 rather than 50 so the binomial success rate has a 95% CI half-width of ≈0.10 rather than ≈0.14 (a compromise short of 200's ≈0.07 at twice the runtime); the per-episode resume carries the sweep across sessions. Planners evaluated: `global_cem`, `local_cem`, `local_gd`, `local_adam`, `hybrid_cem_local_refine`, `hybrid_cem_local_refine_global_rescore`.

8 Results

[TBD: This section will be completed after the first full PointMaze run (surrogate training to 20 000 steps + six-planner evaluation, 100 episodes each).]

8.1 Surrogate training

[TBD: training/validation loss curves; per-horizon validation rollout MSE (steps $1 \dots K$); final *vs-static* ratio. Figures exported by notebook 08 to `_summary/figures/`.]

	one-step MSE	K-step rollout MSE	rollout MSE / static baseline
final validation	**[TBD:]**	**[TBD:]**	**[TBD:]**

Table 2: Surrogate validation summary (placeholder).

8.2 Planner comparison

Planner	succ.	norm. dist. ρ	wall time/ep. (s)	glob. fwd	loc. fwd	bwd steps	accept. rate	disagreement
`global_cem`	·	·	·	·	–	–	–	–
`local_cem`	·	·	·	–	·	–	–	·
`local_gd`	·	·	·	–	·	·	–	·
`local_adam`	·	·	·	–	·	·	–	·
`hybrid_cem_local_refine`	·	·	·	·	·	·	·	·
`hybrid_..._global_rescore`	·	·	·	·	·	·	·	·
reference replay (dataset actions)	–	·	–	–	–	–	–	–

Table 3: Planner comparison on offline latent goal reaching, 100 held-out episodes (placeholder). Distances are judged by the global model (see the stated bias in §5); efficiency columns carry the local/global claim.

[TBD: optimization traces (CEM best cost per iteration; refinement stage); hybrid refinement outcomes (improved / within-tolerance / rejected); analysis of acceptance rate vs. surrogate disagreement.]

8.3 Synthetic-task sanity check (preliminary)

On the synthetic point-mass task — where the global model is exact and the surrogate is trained for only a few hundred steps — the smoke-scale runs (2–3 episodes; not statistics, a correctness signal) reproduce the expected qualitative ordering: all six planners succeed; `local_adam` attains the lowest final distances (a well-trained surrogate plus gradients beats sampling at equal budget); hybrids improve on pure CEM and their refinements are accepted; plain `local_gd` is weakest. In the adversarial unit test where the surrogate's action gain is sign-flipped, refinement reliably worsens the global cost and the re-score gate rejects it, returning the CEM sequence unchanged.

9 Discussion and limitations

What the protocol can and cannot conclude. The offline protocol measures planner behavior *under the global model's belief about the world*. It can establish that first-order refinement through a cheap surrogate recovers (or fails to recover) global-CEM-quality solutions at a fraction of the global-model compute, and how often the surrogate's gradient directions agree with the global model. It cannot, by construction, rank planners against reality: that requires the real-environment evaluation listed below. The reference-replay line partially de-confounds this by exposing the global model's own rollout error on demonstrated behavior.

Surrogate capacity vs. exploitability. The surrogate is deliberately small and its projection frozen; the open question the results must answer is whether PointMaze dynamics in pooled DINOv2 space are smooth enough for useful gradients at this capacity. The Jacobian penalty (λ_J), the GRU context variant, and the grid-vs-mean pooling ablation are the pre-built levers if acceptance rates come back low.

Single seed, single task. The first run is `seed0` on PointMaze only; seed replication and the PushT extension (blocked on a PushT latent cache upstream) are future work by design.

10 Future work

(1) Real-environment confirmation of the planner ranking in MuJoCo PointMaze, reusing the upstream planning path for the global baseline and adding an environment-backed episode runner for local/hybrid planners; (2) seed replication and the built-in ablations (projection, context model, optimizer); (3) FOG-style online surrogate fine-tuning on global-model rollouts to keep local gradients on-distribution under the current action distribution; (4) surrogate ensembles with disagreement penalties inside the planning cost; (5) PushT, where contact dynamics should stress the surrogate and the global re-score safeguard is expected to matter most.

Reproducibility

Code lives in the `wm-prediction` repository (`src/wm_poc/local_global`, `scripts/local_global`, `configs/local_global`); notebook 03 runs the full pipeline on Colab (T4 or A100) with self-gating, resumable stages; `run_smoke.sh` reproduces the synthetic end-to-end check on CPU in under two minutes. Design rationale is documented in `docs/local_global_techniques.md`, the operational guide in `docs/local_global_runbook_next_steps.md`, and the original implementation brief in `LOCAL_GLOBAL_DINO_WM_IMPLEMENTATION_SPEC.md`.

References

[1] P. Amigo, R. Khorrambakht, N. Mansard, L. Righetti. *Coupled Local and Global World Models for Efficient First-Order Model-Based Reinforcement Learning.* arXiv:2602.06219, 2026.

[2] G. Zhou et al. *DINO-WM: World Models on Pre-trained Visual Features Enable Zero-shot Planning.* 2025.

[3] M. Oquab et al. *DINOv2: Learning Robust Visual Features without Supervision.* arXiv:2304.07193, 2023.

[4] D. Hafner et al. *Learning Latent Dynamics for Planning from Pixels.* ICML, 2019.

[5] D. Hafner et al. *Dream to Control: Learning Behaviors by Latent Imagination*. ICLR, 2020.

[6] P. Wu et al. *DayDreamer: World Models for Physical Robot Learning*. CoRL, 2023.

[7] N. Hansen, H. Su, X. Wang. *TD-MPC2: Scalable, Robust World Models for Continuous Control*. ICLR, 2024.

[8] B. Amos et al. *Differentiable MPC for End-to-end Planning and Control*. NeurIPS, 2018.

[9] M. Assran et al. *V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning*. 2025.

[10] *JEPA World Models for Physical Planning*. 2026. (Full citation to be completed.)